

阅读文献笔记(9.25-9.27)

1 关于 SGLang（一种作用与模型的推理引擎）

(1)基数树 Radix Tree:

基数树是一种压缩前缀树。拥有相同 token 前缀的序列会共享树的上层节点。

树上每个节点，保存一段连续 token 序列 + 这段 token 对应的 KV 缓存

当新 prompt 进来，顺着树遍历，找到最长可以匹配的前缀节点，直接复用该节点 KV。

(2)关键内容：RadixAttention,缓存调度感知，注意力计算，SGLang 结构化解码

RadixAttention

它会把所有请求的 KV 缓存组织成一个基数树，树上的每个节点都存储一段 token 序列以及对应的 KV 缓存。

第一步（新请求进入），它会把 prompt 转为 tokenx 序列，然后遍历基数树，寻找当前 prompt 能够匹配的最长前缀

第二步（命中前缀），读取基数树上已经存在的 KV 缓存，然后跳过这部分对应的 prefill，只对剩下不匹配的 token 做 prefill,生成新 KV 节点，然后插入基数树中。

第三步(全都没命中),这时就会让整段 token 序列进入 prefill,生成 KV 缓存后插入基数树上

第四步（请求结束），KV 缓存插入基数树后不会被释放，它会保存下来以供后续其他请求复用

第五步(淘汰旧 KV)，这一步只发生在 GPU 显存满了的时候，它会淘汰掉好久没用过的 KV 节点

缓存感知调度（Cache-aware Scheduling）

SGLang 调度器会优先调度可以命中 KV 缓存的请求，提升缓存命中率，进一步减少 Prefill 计算，降低 TTFT。

注意力计算

RadixAttention 复用 KV 缓存之后，注意力计算直接读取树上已经存在的 KV，不需要重复计算。底层同样支持分页 KV 内存管理，消除内存碎片。

SGLang 结构化解码（压缩 FSM）

针对 JSON、工具调用等这类结构化输出场景，普通生成会产生非法格式文本，需要额外校验。所有，这项功能人 SGLang 能用压缩有限状态机(FSM)约束输出 token,让它只生成符合格式的 token,减少无效采样，提升速度。

(

什么是结构化输出场景？

普通对话：模型随便写自由文本，想写多长、用什么句式都可以（非结构化输出）。
->弊端：模型很容易生成非法 JSON，比如少一个引号、少括号，下游程序解析失败，需要反复重试，浪费算力、增加延迟。

结构化输出：强制模型输出符合指定语法规则的文本，格式错了就不能用

故，上述所说的普通生成会产生非法格式文本，就是指普通对话，没用固定格式,容易出错

)

2.关于 vllm（和 SGLang 一样是推理引擎）

Vllm 出现之前的问题:

LMM 应用成本高昂，L 每一条请求的开销大,一条是传统数据库的 10 倍以上;

传统的批处理方式会按照最大可能长度为 KV 分配内存，导致严重的内存浪费。更糟糕的是，KV 缓存占内存太大，频繁分配和释放大块连续内存会产生大量的内存碎片与数据冗余，随着服务持续运行，显存中会出现很多零散的空闲小块，造成没有连续空间能够容纳新请求，系统触发 OOM(显存不足).这样，内存碎片限制了批处理规模，即运行速度被限制住了;

vllm 效果:与 FasterTransformer 和 Orca 相比，提高了 2~4 倍的吞吐量；相同延迟下吞吐高，相同吞吐下延迟低；规模越大性能提升效果越好；

vllm 的解决方案->PagedAttention

原理： 基于操作系统的虚拟内存分页机制；将 KV 分成固定大小且相互独立的页，这样每个 KV 就是动态且可以任意释放的，我们只需要维护一张页表用来记录每个请求对应的 KV 位置即可。

核心优势:

- a. 消除了内存碎片；内存分配只需要找空闲的单页，不再需要大块连续内存；释放时只要释放各个页面，不会产生零散的空闲小块；
- b. 支持 KV 的页级共享；多条请求如果拥有相同 prompt 前缀，它们的 KV 就可以被其他请求共享，大幅度节省内存。

连续批处理

无需等待多条请求在的那一批次全部完成，任意一条请求被完成，可以马上跑下一条；但这需要频繁地创建和销毁请求（内存），对 KV 缓存内存管理有很高要求，内存碎片地问题会被放大（PagedAttention 解决了这个，因此连续批处理实现地很成功）

Vllm 系统架构

基于 PagedAttention 和连续批处理，

被分为调度器、GPU 工作线程、页表内存管理器

调度器： 接受请求，管理请求列队，执行批处理；一个请求完成后，调度器会把新请求送入 prefill 或者 decode 阶段

GPU 工作线程： PagedAttention 定制 CUDA，能够读取离散的 KV 页面，完成注意力计算。（就是让 KV 能够离散地存储，不用弄成连续的内存）

页表内存管理器：负责页面的分配、释放、引用计数，管理 KV 页面共享，维护所有页表

整个流程:

- a. 用户提交 prompt；调度器接收，tokenizer 转 token 序列。
- b. 内存管理器分配页表，调度器送入 Prefill 批次，一次性计算 prompt 全部 token，生成 KV 页面。
- c. Prefill 完成后进入 Decode 循环：每次生成一个 token，分配新 KV 页面，更新页表。

d. 遇到终止符 eos，请求结束；内存管理器根据页表，释放该请求的 KV 页面（共享页面只减少引用计数）。

3.关于 Mooncake

区分 vllm 和 Mooncake:

Vllm 是推理引擎，它是单台机器 GPU 显存池内的 KV 缓存共享

Mooncake 不是推理引擎，它是一套分布式 KV 缓存池+prefill/Decode 解耦的系统架构，可以外接 vLLM 这样的推理引擎，做到跨机器、持久化的全局 KV 前缀复用

之前的问题:

vLLM / SGLang 这类推理引擎，KV 缓存生命周期绑定请求；请求结束，KV 立即释放。PagedAttention/RadixCache 仅支持同批次内短暂共享，无法跨请求持久复用。

Prefill 与 Decode 混部在同一 GPU，两种任务资源争抢：Prefill 是高算力并行任务，Decode 是轻量串行任务，混在一起会互相干扰，降低 GPU 利用率

Mooncake 的效果:

- a. 吞吐量：提升 3.8 倍；
- b. TTFT（首 token 延迟）P50：降低 46 倍；
- c. 端到端延迟：降低 8.6 倍；
- d. KV 缓存命中率最高可达 92.2%；
- e. 在长上下文场景优化收益更加明显

Mooncake 架构:

整体分为三大组件：Prefill 集群、Decode 集群、Mooncake Store 全局 KV 缓存池

a. Prefill 集群：接收新用户请求，首先计算请求前缀的块哈希。

如果全局缓存命中该前缀 KV：直接取出已有的 KV，只对新增文本做 Prefill；

如果未命中：完整执行 Prefill，生成 KV 缓存写入 Mooncake Store。

b. Mooncake Store（全局 KV 缓存池）：分布式分层存储，支持 GPU 显存、DRAM、SSD。保存 KV 块，使用块哈希索引；支持 KV 缓存跨请求持久复用。采

用引用计数管理 KV 块生命周期，当没有任何请求引用 KV 块时才回收。

- c. Decode 集群：从 Mooncake Store 读取 KV 缓存，执行 Decode 循环生成输出 token。Decode 集群只负责解码，不需要做 Prefill heavy 计算。
- d. 调度器：路由请求，管理块哈希查找，维护 KV 缓存引用计数，协调 Prefill/Decode 集群。

块哈希机制 (Block Hash)

把 prompt 切分为固定长度块，每一块计算哈希值。新请求到来时，按块依次比对哈希：快速找到最长可复用前缀。

请求完整流程：客户端发送 prompt → 调度器计算块哈希 → 查询 Mooncake Store

若命中，直接复用已有 KV，仅对新增 token Prefill → Decode 读取 KV 生成 token，流式返回给用户；若未命中，就进行完整 Prefill 计算，KV 存入 Mooncake Store → Decode 生成 token